

Predicting Next-Day S&P 500 Stock Movements: Best ML/DL Models

Overview

Predicting whether an S&P 500 stock will go **up**, **down**, or **remain stable** on the next trading day is a challenging task. Modern approaches integrate multiple data sources – **fundamental indicators**, **news sentiment**, and **current price/technical data** – to capture diverse drivers of price movement. Recent research and competitions emphasize *raw predictive performance* (accuracy, precision/recall, etc.) and **real-world trading metrics** (like portfolio returns or Sharpe ratios) rather than interpretability. Below, we summarize the best-performing machine learning and deep learning models for this task, typical data preprocessing pipelines, and relevant benchmarks. We focus on models feasible to train on a consumer GPU (e.g. RTX 3060–4090) within ~12 hours, using U.S. stock data.

Top Models and Architectures for Stock Movement Prediction

Ensemble Tree Models (Random Forests & XGBoost)

Description: Tree-based ensemble methods have been strong performers in many stock prediction studies. Techniques like Random Forest (RF) and **XGBoost** (gradient-boosted trees) handle tabular features well and are relatively lightweight to train. They excel at combining disparate features – e.g. fundamental ratios, sentiment scores, technical indicators – into a single predictive model. In one large-scale study, ensemble methods (RF, XGBoost, Extra Trees, etc.) were applied to thousands of stocks; an Extra Trees classifier achieved the highest average F1-score across stocks ¹ ². Similarly, an ensemble of RF, boosted trees, and a neural net outperformed any single model in predicting 1-day S&P 500 returns ³. Kaggle’s 2019 Two Sigma competition (using news + market data) saw many participants rely on ensemble models due to their robustness under limited training time; one silver-medal solution used an Extra Trees (RF) model and found that *news sentiment features did not significantly improve accuracy* ⁴. (Many top competitors simply ignored the news data, focusing on price signals, likely to avoid overfitting noise ⁴.) Overall, tree ensembles often serve as a **strong baseline** in this domain.

- **Pros:** Fast training and easy to tune (runs comfortably on a laptop GPU/CPU); handles **mixed data** types and feature engineering easily; less prone to overfitting than deep nets on small data; yields feature importance for analysis; has delivered solid accuracy and even trading profits in research ⁵ ⁶.
- **Cons:** Does not natively model sequential time dependencies – requires manual creation of lagged features or technical indicators; may miss complex patterns captured by deep networks; typically needs sufficient feature engineering (the model won’t “discover” temporal patterns without human-defined inputs). Also, ensemble methods capturing small edges can be susceptible to regime changes (they might overfit to past regimes unless updated frequently).

Recurrent Neural Networks (LSTM/GRU Models)

Description: Long Short-Term Memory (LSTM) networks and related RNNs are popular for time-series stock prediction. They “remember” prior timesteps, making them well-suited to modeling sequences of prices and news sentiment over time. LSTMs have demonstrated leading performance in academic and real-world tests. A landmark study by Fischer & Krauss applied an LSTM to all S&P 500 constituents (1992–2015) to predict next-day up/down moves ⁵. The LSTM significantly outperformed memory-free models (RF, deep feed-forward network, logistic regression), achieving an **out-of-sample daily return of ~0.46% (Sharpe ~5.8)** before transaction costs ⁵. (This corresponds to about 52–53% directional accuracy – enough edge to produce high Sharpe when trading many stocks daily.) Subsequent research found LSTM classifiers consistently beat traditional ML in this domain: for example, using 2017–2022 data on all S&P 500 stocks, an LSTM model had higher accuracy/precision than Support Vector Machines or Random Forests in predicting which stocks would outperform the index, with results very unlikely due to chance ($p \sim 8e-17$) ⁷ ⁸. LSTMs also appear better at **integrating news sentiment signals** than simpler models. A 2025 study on energy sector stocks showed that adding FinBERT-based news sentiment features significantly improved an LSTM’s prediction accuracy, whereas an XGBoost model **gained no benefit from sentiment** (its best performance was using price features alone) ⁹. In short, LSTMs can learn subtle temporal patterns (e.g. short-term mean-reversion, momentum bursts, sentiment impacts) that boost predictive power for next-day moves.

- **Pros:** Captures temporal dependencies in price and news data (e.g. can learn that “if stock dipped and news sentiment is rising, a rebound is likely”); has achieved **state-of-the-art accuracy and trading returns** in research ⁵; handles multivariate sequences (price, volume, sentiment time series) in a unified network. LSTMs are reasonably lightweight – e.g. a 3-layer LSTM with a few hundred units can be trained on years of daily data in minutes to hours – within the 12h constraint. They often outperform static models like RF when patterns unfold over time ¹⁰ ⁸.
- **Cons:** Requires more data and tuning to avoid overfitting. Daily stock data are relatively limited (only ~252 trading days/year), so a deep LSTM might overfit without regularization or multi-stock training. Training is slower than tree models, especially if tuning many hyperparameters. Also, **market patterns can change** – one study found the LSTM’s trading advantage largely vanished after 2010 as the learned “alpha” (short-term reversal on volatile stocks) was arbitrated away ¹¹. This means an LSTM may latch onto patterns that don’t persist, so real-world performance can decay without model updates. Finally, interpretability is low (though not a priority here), and debugging an LSTM can be tricky if it’s not performing.

Transformer-Based Models

Description: Transformers have emerged as powerful sequence models, offering the ability to capture long-range dependencies through self-attention. In financial applications, Transformers are used in two ways: (1) **Text/Sentiment Models** – pre-trained language models like BERT or FinBERT to extract sentiment/features from news; and (2) **Time-Series Models** – Transformer architectures applied to price or multi-modal sequences. A prime example is **FinBERT**, a BERT variant trained on financial text. FinBERT can turn news headlines or articles into granular sentiment scores. Incorporating these into a predictive model has shown remarkable gains. A recent multimodal framework (2018–2023, S&P 500 stocks) fused FinBERT-extracted news sentiment with technical indicators and fed them into an ensemble model (XGBoost). The **FinBERT-enhanced model** consistently outperformed a technical-only baseline, improving AUC by ~12.6% and even boosting average backtest P&L by 26% ¹². It achieved higher precision (0.678 vs 0.601) in predicting upward moves, meaning far fewer false-positive trades ¹². These results highlight that

Transformer-derived features (FinBERT) were the top predictive contributors – more influential than any price-based feature, as confirmed by SHAP importance analysis ¹³ ¹⁴ . Beyond text, researchers have applied sequence Transformers (and hybrid variants like Temporal Fusion Transformer) directly to market data. Transformers can ingest price history, indicators, and even multiple stocks' data in parallel. They excel at capturing long-term context (e.g. months of trend, or cross-correlations) that an LSTM might miss ¹⁵ . For example, some studies modify attention to focus on key events (earnings dates, etc.), or combine CNNs with Transformers to capture both short and long-term patterns ¹⁶ . In practice, a full Transformer model for daily stock moves is heavy – but feasible on a modern GPU if the dataset is limited to S&P 500 scope. Often, a compromise is to use Transformers for feature extraction (like FinBERT for sentiment, or an encoder for technical series) and then a simpler model (like XGBoost or an MLP) for the final prediction ¹⁷ .

- **Pros: Rich feature extraction** from text via NLP Transformers – this has proven to significantly improve predictive accuracy and trading outcomes ¹³ ¹² . Transformers can model **long-range patterns** (weeks or more of data) better than RNNs, which is useful if the prediction relies on trends or persistent fundamental signals. They are also flexible: one Transformer model can combine heterogeneous inputs (prices, volumes, multiple related stocks' data, news text) by design. Cutting-edge approaches even use large language models to **generate new features (“alpha factors”)** which are then used by a Transformer predictor ¹⁸ ¹⁹ – an innovative way to inject human-like pattern ideas into the model. With transfer learning (pre-trained models), one can leverage massive text corpora – e.g. FinBERT already knows financial language nuances, so it yields more accurate sentiment than generic methods ²⁰ ¹⁴ . Overall, transformer-based pipelines have achieved **state-of-art results** in combining sentiment with technical data for stock direction forecasting in recent studies ¹³ ¹² .
- **Cons: Computationally intensive** – full Transformers require more memory and time. Training a large Transformer from scratch on a single laptop GPU may be infeasible under 12 hours, though fine-tuning a pre-trained FinBERT on your news data or using a moderate-sized transformer for prices is possible. The complexity also means more risk of overfitting if data is limited (daily data for 500 stocks is not “big” by deep learning standards). Another practical concern: improvements from Transformers might plateau unless you have a lot of data or very relevant features – for example, simply feeding raw price series into a transformer might not greatly beat an LSTM unless you include additional context (multi-stock data, events, etc.). Additionally, when evaluating in real trading, one must account for execution costs – the FinBERT study's P&L assumed ideal execution ¹² , so real returns would be lower once transaction costs/slippage are considered. In summary, Transformers are powerful but **resource-heavy**, and one must ensure the gain in predictive edge justifies the added complexity for a deployable trading model.

Graph Neural Networks & Hybrid Models

Description: Graph Neural Networks (GNNs) are a newer class of deep models that can learn from relational data – in this case, relationships between stocks. The S&P 500 stocks form a network (by industry, supply chain links, correlations, etc.), and GNN-based models leverage this structure to improve predictions. For example, a hybrid **LSTM-GNN** model can use an LSTM to capture each stock's time-series dynamics and a GNN to share information between related stocks ²¹ ²² . If Apple and Microsoft have strongly correlated movements, a GNN can inform Apple's predictor with Microsoft's recent trends (and vice versa). A 2025 study demonstrated that such a hybrid LSTM-GNN *“significantly outperformed baseline models”* that lacked one of those components ²³ . Similarly, advanced frameworks like **DASF-Net** construct graphs based on both industry sector and historical return correlations among stocks, then diffuse information over the graph each day. By fusing this with news sentiment signals, DASF-Net achieved lower error and higher

robustness in forecasts compared to simpler LSTM or CNN-based models ²⁴ ²⁵ . The intuition is that **market moves are contagious**: news about one company can affect its peers, and sector trends or index movements create latent factors that a single-stock model might miss. GNNs explicitly model these effects. Another approach is to use knowledge graphs or co-mention networks from news: e.g., linking companies that appear in the same news story and propagating sentiment through that network. Recent work incorporated “*related companies’ sentiment*” (companies frequently mentioned together) into a transformer model and found it improved accuracy ²⁶ ¹⁹ . Overall, GNNs and hybrid models are cutting-edge and show promise in academic settings for squeezing out extra predictive power by modeling the market as a system of interconnected parts.

- **Pros:** Captures **cross-stock relationships** that other models ignore. This is important in an index like S&P 500, where stocks aren’t independent – industry trends, macro news, or fund flows cause group movements. A GNN can learn that structure (e.g. “bank stocks move together”) and use it to improve predictions, especially for the *up/down “stable” classification* of each stock. Indeed, combining temporal and relational learning yielded a “significant improvement in predictive accuracy” in experiments ²⁷ . GNNs can also effectively increase the data used for each prediction by letting each stock’s model learn from others’ data, which is useful under data constraints. They remain feasible on a single GPU for S&P 500 scale: 500 nodes is a small graph, so even a complex GNN+LSTM hybrid can train in hours with modern libraries. These models often outperform purely sequential models when evaluated on real-market datasets ²³ , and they can be evaluated in backtests similarly (e.g., by generating signals for many stocks).
- **Cons: Complexity:** implementing and tuning hybrid LSTM-GNN or GNN-Transformer architectures is more involved. There are more moving parts (graph construction, choice of graph features, etc.) and potential for bugs or overfitting. The data pipeline needs additional steps to build the relationship graph (e.g., computing correlations or sourcing knowledge graph data). If the graph relationships are mis-specified or change over time, the GNN could add noise. From a compute perspective, while moderate GNNs are okay, very deep or dynamic graph models could push memory/time limits. Also, GNNs in finance are still an emerging area – improvements over simpler models, while evident in some research, may not always justify the added complexity in a live trading context. Under the 12-hour training constraint on a laptop, you likely can train a moderate hybrid model, but extensive hyperparameter searches (for number of GNN layers, etc.) might be limited. Finally, like other deep models, GNNs can suffer if market regime shifts – relationships that held in training may break down (e.g., correlations between stocks can invert in a crisis), so any fixed graph needs periodic updating.

Other Notable Approaches

In addition to the above, researchers have experimented with other architectures: **Convolutional Neural Networks (CNNs)** (e.g. treating sequences of technical indicators as “images” or applying 1-D CNN on price series) and deep **feed-forward networks** on engineered features. CNNs have shown some success, especially when combined with attention mechanisms for text data ²⁸ or when used in hybrid with LSTM (CNN to extract short-term patterns, LSTM for longer-term) ²⁹ ³⁰ . Generally, pure CNN or MLP models alone have not surpassed LSTMs for sequence modeling, but they can be part of an ensemble or hybrid. Also, simpler **logistic regression or SVM** models are sometimes surprisingly competitive when given good features, though they usually underperform tree ensembles or deep networks on nonlinear financial patterns ³¹ . In practice, many winning solutions and papers use **ensembles of multiple model types** (to hedge against model-specific errors). For example, combining an LSTM with an XGBoost or averaging an RNN, a CNN, and a GNN model – these can improve stability of predictions at the cost of training time.

However, on a single GPU with limited time, one might prioritize a single strong model rather than a large ensemble.

Data & Preprocessing Pipeline

Feature Construction is crucial when combining fundamentals, sentiment, and price data:

- **Fundamental Data:** Fundamental indicators (P/E, P/B, earnings growth, etc.) for S&P 500 companies are typically updated quarterly. For daily prediction, a common approach is to treat the latest known fundamentals as static features attached to each day's input. For example, you might use the last quarter's financial ratios, analysts' ratings, or sector averages as part of the feature vector. These should be **normalized** (e.g. z-scored or rank-transformed within the universe) to prevent scale differences from skewing the model. Some studies also use fundamental-derived signals like *"earnings surprise"* or *"recent earnings revision sentiment"* as features, which can help if a stock's fundamentals changed recently. Overall, fundamental features provide context but change slowly – indeed one experiment found technical price trends outweighed pure fundamental ratios for short-term prediction ³² ³³. Thus, fundamentals might help more for filtering long-term winners vs losers, while for day-to-day direction their contribution is often modest.
- **News Sentiment:** To incorporate news, the pipeline usually involves **natural language processing** to convert text into numeric sentiment scores. A state-of-the-art method is using FinBERT or similar financial news models to get sentiment probabilities (positive/neutral/negative) for each news headline or article ²⁰ ¹⁴. These can be aggregated by day and by stock: e.g. compute the daily average sentiment, the max/min sentiment, and counts of positive vs negative news for each stock. (The FinBERT-based study above used daily mean sentiment, sentiment volatility, and extreme sentiment values as features ³⁴ ³⁵.) If using a simpler approach, one might use a lexicon (e.g. Loughran-McDonald word lists or VADER sentiment) to score news, but note that lexicon methods underperform FinBERT – in one evaluation, a VADER-based sentiment model lagged far behind FinBERT in accuracy and P&L ³⁶. **Preprocessing:** You'll need to map each news item to the related stock(s) (usually by ticker or company name mention). Ensure that at prediction time, you only use news that was published *before market close* of the current day if predicting next day – to avoid lookahead bias. Some pipelines include not just the target stock's news but also news of **related companies** (e.g. suppliers, competitors) as features ²⁶, since those can have spillover effects. After computing sentiment features, you may standardize them (e.g. subtract mean, divide std) across the training set. Many studies align news sentiment by date with end-of-day prices; intraday timing can be considered (e.g. news after 4pm might only affect next trading day), but for daily granularity a common simplification is to aggregate all news from the last 24 hours.
- **Price & Technical Features:** These include the stock's recent returns and technical indicators that capture momentum, volatility, volume patterns, etc. A typical preprocessing step is to calculate **percentage returns** from prices (this stationarizes the series). Many implementations feed the raw closing price sequence (normalized) into an LSTM or transformer, sometimes along with technical indicator time series like moving averages. An efficient representation is to create a fixed-length window of recent daily returns as the input sequence (for example, the past 60 trading days) ³⁷. You can also add **technical indicators** as extra channels or features for each day – common ones: moving average crossover signals, RSI, MACD, Bollinger Bands, trading volume changes, etc. Including volatility measures (e.g. 5-day standard deviation of returns) is helpful since volatility

regime can change model behavior ³⁸. In fact, one Kaggle strategy was to **exclude high-volatility stocks** from trading because they hurt Sharpe ratio despite prediction accuracy ³⁹. If doing classification (up/down/stable), you need to define the “stable” threshold – e.g. treat movement within $\pm 0.1\%$ or $\pm 0.5\%$ as the stable class. This can help by not forcing a decision on days with insignificant change. Ensure your class labels are balanced or use techniques (like more data or class weights) if one class is rarer. Lastly, apply feature scaling to numeric inputs if using neural networks (e.g. Min-Max scale prices, or simply feed returns which are already percentage scale). The Htun et al. (2024) study, for example, formed feature vectors of **13 lagged relative returns** (1-day, 5-day, 10-day, ... up to 1-year) for each stock ⁴⁰ ³¹. This is a concise way to capture multi-scale momentum in a format suitable for an RF or even a dense NN. For an LSTM, one could feed the sequence of daily returns directly.

- **Data Splitting and Validation:** Because stock data are time-dependent, we use *time-series splits* (train on past, test on future data). It’s critical to avoid shuffling or any lookahead. Common practice is using the most recent segment of history as a test set (e.g. train on 2015–2020, test on 2021–2022). Rolling cross-validation is also used: train up to month X, test on month X+1, and repeat, to assess stability. In research, metrics like accuracy, F1, Matthews correlation, and AUC are reported, but **real trading evaluation** involves backtesting a strategy based on the model’s predictions. For example, a model might rank stocks by predicted probability of “up” and then a portfolio goes long the top decile and short the bottom decile each day – measuring resulting returns and Sharpe. The Kaggle Two Sigma competition explicitly used a weighted Sharpe ratio of the daily returns of a hypothetical portfolio as the metric ⁴¹. In our context, after building a model, one should simulate trading: e.g. buy if model says “up” with high confidence, short if “down”, perhaps hold cash if “stable.” This will reveal if the model’s accuracy translates to profits after transaction costs.

Benchmarks and Real-World Performance

Academic Benchmarks: Numerous studies have evaluated these models on historical market data. As mentioned, Fischer & Krauss (2018) is a notable benchmark on S&P 500 stocks, where an LSTM achieved **~53% directional accuracy** and significant abnormal returns, beating logistic regression, random forest and a deep neural net ⁵ ⁴². Another comprehensive study (Htun et al. 2024) tested RF, SVM, and LSTM classifiers on all S&P 500 stocks (2017–22) to predict 10-day ahead outperformance. The LSTM was the best, with statistically higher accuracy, precision, recall than RF or SVM in aggregate ⁸ ⁴³. These works give a sense that a well-tuned ML/DL model can **beat random guessing by a meaningful margin** (often 55–60% accuracy in classifying up vs down), which in turn can underpin profitable trading strategies. Another benchmark: Krauss et al. (2017) used a **gradient boosted trees, RF, and neural nets ensemble** on S&P 500 daily data, reporting ~51-52% accuracy for individual models and ~53% for the ensemble, and substantial portfolio gains ⁴⁴. This highlights that ensembles can squeeze out extra performance ³.

Competitions: The **Kaggle Two Sigma: Using News to Predict Stock Movements (2019)** is a relevant benchmark competition. It provided news sentiment features (from Reuters) and market data for US stocks (including but not limited to S&P 500). The goal was to predict a *confidence* value for each stock’s next-day return, maximizing Sharpe ratio of a simulated portfolio. The top methods were not fully public, but participants reported that simpler models often worked best under the competition’s constraints. For instance, the 145th place (silver medal) approach used an ExtraTrees ensemble and noted “the news data never seemed to contribute much. Many competitors didn’t even use it.” ⁴. Instead, controlling risk was vital – top teams constructed portfolios that excluded volatile stocks and sized positions conservatively to

juice the Sharpe ³⁹. This teaches that the *most accurate model* is not automatically the *best trading model* unless you account for volatility and correlations in predictions. Nonetheless, the competition advanced research in combining alternative data with prices. Another competition, **Numerai**, continuously crowdsources stock prediction models (on a meta level) – many winners have used variations of tree ensembles and deep nets, but direct comparison is hard since data is obscured and not specific to S&P 500.

Real-world trading: Ultimately, a model's value is measured by live or backtested trading performance. It's encouraging that many of the cited studies demonstrate positive backtest results. The FinBERT+XGBoost model, for example, showed that including news sentiment could improve a strategy's average P&L by ~26% over a baseline (in a simulation) ¹². Fischer & Krauss's LSTM strategy yielded huge returns pre-2010, though it flatlined after (reminding us that markets adapt) ¹¹. When evaluating these models, one should do an out-of-sample backtest with realistic assumptions (transaction costs, market impact for large caps are low but non-zero). A model with even 0.5% average daily return before costs (like Fischer & Krauss's) might drop to 0.2% after costs – still lucrative with leverage. On the other hand, models that look good on paper can fail in live trading due to *overfitting* or regime change. A common practice is **walk-forward testing** (periodically retraining on expanding data and testing on the next period) to mimic live deployment; this often gives a more sober picture of performance. As an example, one study using LSTM on Chinese stocks reported ~54% accuracy which led to significant excess returns ⁴⁵ – similar performance in the US context might yield a Sharpe in the 2–4 range in backtests, which is excellent. There have also been **industry applications** where hedge funds and prop traders use these models. Two Sigma (the sponsor of the Kaggle contest) presumably integrated news sentiment in some of their quantitative strategies. J.P. Morgan AI researchers have published on using transformers for market risk and signal extraction ⁴⁶, indicating growing industry adoption.

In summary, **best-performing models** in this domain tend to be **hybrid** and **data-intensive**: e.g. an LSTM or Transformer that ingests price history along with sentiment features, possibly augmented by an ensemble or GNN to integrate cross-sectional info. Under hardware/time constraints, a reasonable approach is to use a gradient boosting model or a moderately deep neural network that combines engineered features (lagged returns, technicals, latest fundamentals, sentiment scores). Such models have achieved **60–65% accuracy in short-horizon direction prediction** when carefully built ⁴⁷ ⁴⁸, and more importantly, they have shown tradable performance when evaluated via backtesting (with some delivering Sharpe ratios above 2 or 3 before costs ⁵).

Pros and Cons Summary by Approach (Under Constraints)

- **XGBoost / Random Forest: Pros:** Easy to train on a local machine; handles many feature types (fundamental ratios, sentiment scores, etc.) with minimal preprocessing; robust to overfitting if tuned; fast to update with new data. Often a **top performer on tabular financial data**, sometimes rivaling deep networks in accuracy ⁹. **Cons:** Needs good feature engineering (cannot on its own model time sequence dependencies or extract text sentiment without precomputed features); may not capture subtle temporal patterns that RNNs or Transformers can; typically provides smaller incremental improvements once basic features are in place.
- **LSTM / RNN: Pros:** Naturally models sequential data – effective for capturing short-term trends, mean-reversion, and the effect of news over several days. Demonstrated **higher predictive accuracy than static models** in many studies ⁸. Feasible to train within hours on GPU for S&P 500 (especially if using mini-batches across stocks). Can ingest sequences of technical indicators and

sentiment trend over time, potentially improving signal detection. **Cons:** Computationally heavier than trees – tuning many hyperparameters (layers, units, lookback window) can eat up time. Risk of overfitting given limited samples per stock; requires techniques like dropout or early stopping. The “black box” nature can make it hard to debug when performance shifts. Also, model needs to be retrained regularly to keep up with regime changes – an LSTM trained on 2010–2020 data might perform poorly in 2021 if new patterns emerge (thus requiring frequent retraining, which is manageable but must be noted).

- **Transformers: Pros:** State-of-the-art sequence modeling; **excels at incorporating long-term context and multi-modal data.** Using pre-trained models (like FinBERT) massively boosts the model's understanding of news language, directly yielding accuracy gains and higher trading profits in studies ¹³ ¹² . A Transformer-based architecture can flexibly combine time-series, text, and even cross-sectional info, which is hard to do with other models. **Cons:** High resource requirement – training a transformer from scratch on price data might not be optimal on a laptop (though not impossible with a small model and short sequences). In practice, one might use a transformer for feature extraction (which can be done in a reasonable time) and not train the whole thing end-to-end on the laptop. Also, Transformers demand careful hyperparameter tuning (learning rates, attention heads, etc.), which can be time-consuming. Another con is that they are prone to overfitting if the dataset is not sufficiently large or if regularization like dropout and weight decay are not used liberally. Given the 12h constraint, one might use a **pre-trained FinBERT for sentiment** (leveraging OpenAI/Transformers libraries on GPU to process news) and then feed those features into a simpler model – this yields much of the benefit without the need to train a giant model from scratch ¹⁷ .

- **Graph Neural Networks / Hybrids: Pros:** Capture relationships between assets, potentially improving predictions for stocks influenced by common factors. GNNs can **learn sector or network effects automatically**, which is a big plus in a market where stocks influence each other. They've shown improved accuracy in research settings and can enhance robustness (e.g. if one stock has little data, the GNN can borrow signal from peers). **Cons:** More complex to implement and debug. Need to construct and possibly update the graph (for example, correlation networks might be recomputed periodically). Training a combined GNN+LSTM might require more memory – but since S&P 500 is moderate scale, it's still doable on a high-end consumer GPU. With limited time, the added benefit of a GNN might be marginal unless the relationships are strong and the model is well-tuned. Also, if the focus is one stock at a time, GNNs won't help; they shine when predicting many stocks together (which is our case, all S&P 500). For a single laptop setup, one might simplify by using industry sector dummy features or index features as a proxy for a full GNN (less elegant but very light), if GNN implementation is a bottleneck.

In conclusion, there is **no one-size-fits-all model**, but several promising candidates. For a practitioner with a gaming laptop, a reasonable plan is to start with a powerful yet tractable model like an **XGBoost ensemble using engineered features (including FinBERT sentiment)** – this offers a strong baseline with high accuracy ¹³ ¹⁷ . From there, one could experiment with an LSTM or Transformer encoder to see if sequence modeling of the same features yields further gains. In research literature, **hybrid models** (combining techniques) tend to win, so an ideal solution might blend these approaches: e.g. use an LSTM to encode price history, use a Transformer or BERT model to encode news, and merge them in a dense layer or ensemble. Under the given constraints, focus on models that maximize predictive power per unit of complexity – many sources indicate that **a few well-chosen features (especially sentiment) with a reasonably tuned model can outperform extremely complex architectures** ⁹ ⁴ . The key is rigorous

validation with backtesting to ensure the model's edge is real and consistent. By leveraging the best practices from these studies and competitions, one can identify a model that is both **effective in prediction accuracy and viable in real-world trading**.

Sources: The information above is drawn from a range of recent studies and evaluations, including large-scale experiments on S&P 500 stocks ⁵ ⁷, Kaggle competition insights ⁴, and advanced research integrating news sentiment and market data ¹³ ¹², among others. Each model's described performance and characteristics are supported by these references to ensure factual accuracy and relevance to live trading scenarios.

¹ ² ⁶ ³² ³³ (PDF) Forecasting relative returns for S&P 500 stocks using machine learning
https://www.researchgate.net/publication/379973885_Forecasting_relative_returns_for_SP_500_stocks_using_machine_learning

³ ⁷ ⁸ ¹⁰ ³¹ ⁴⁰ ⁴³ ⁴⁴ Forecasting relative returns for S&P 500 stocks using machine learning | Financial Innovation | Full Text
<https://jfin-swufe.springeropen.com/articles/10.1186/s40854-024-00644-0>

⁴ ³⁹ two sigma | Stephen Smith's Blog
<https://smist08.wordpress.com/tag/two-sigma/>

⁵ ¹¹ ⁴² Deep learning with long short-term memory networks for financial market predictions
<https://ideas.repec.org/a/eee/ejores/v270y2018i2p654-669.html>

⁹ ³⁷ Does business news sentiment matter in the energy stock market? Adopting sentiment analysis for short-term stock market prediction in the energy industry
<https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2025.1559900/pdf>

¹² ¹³ ¹⁴ ¹⁷ ²⁰ ³⁴ ³⁵ ³⁶ ³⁸ Stock Price Prediction Using FinBERT-Enhanced Sentiment with SHAP Explainability and Differential Privacy
<https://www.mdpi.com/2227-7390/13/17/2747>

¹⁵ ¹⁶ ¹⁸ ¹⁹ ²⁶ Sentiment-Aware Stock Price Prediction with Transformer and LLM-Generated Formulaic Alpha
<https://arxiv.org/html/2508.04975>

²¹ ²² ²³ ²⁷ Stock Price Prediction Using a Hybrid LSTM-GNN Model: Integrating Time-Series and Graph-Based Analysis
<https://arxiv.org/html/2502.15813v1>

²⁴ ²⁵ DASF-Net: A Multimodal Framework for Stock Price Forecasting with Diffusion-Based Graph Learning and Optimized Sentiment Fusion
<https://www.mdpi.com/1911-8074/18/8/417>

²⁸ Attention-augmented hybrid CNN-LSTM model for social media ...
<https://www.nature.com/articles/s41598-025-18245-x>

²⁹ ³⁰ Stock Market Prediction Using Machine Learning and Deep Learning Techniques: A Review
<https://www.mdpi.com/2673-9909/5/3/76>

⁴¹ Enhancing Statistical Significance of Backtests
<https://epchan.com/img/links/What-we-learned-from-Kaggle-Two-Sigma-News-Sentiment-competition.pdf>

45 Forecasting stock prices changes using long-short term memory ...

<https://www.nature.com/articles/s41598-023-50783-0>

46 Using FinBERT and temporal fusion transformer to multimodal data ...

<https://www.sciencedirect.com/science/article/abs/pii/S1062940825000014>

47 48 S&P 500 Stock Price Prediction Using Technical, Fundamental and ...

<http://ui.adsabs.harvard.edu/abs/2021arXiv210810826Z/abstract>